

Reliable-Data-Split (RDS): Maximizing Model Potential with Reinforced Selection Strategy

Hoang D. Nguyen

University College Cork, Ireland

HN@CS.UCC.IE

Xuan-Son Vu

Umeå University, Sweden

SONVX@CS.UMU.SE

Quoc-Tuan Truong

Singapore Management University, Singapore

QTTRUONG.2017@SMU.EDU.SG

Duc-Trong Le

Vietnam National University, Vietnam

TRONGLD@VNU.EDU.VN

Abstract

The nexus between data characteristics and parametric models is fundamental for developing effective and reliable artificial intelligence (AI) systems. Mismatches in data properties for model development may lead to deleterious effects on AI model performance in machine learning practice. This paper proposes a Reliable Data Split (RDS) procedure to learn how to select data points that will generalise the target domain adequately by employing prior knowledge of the data generative process. We introduce a reinforced selection strategy using deep reinforcement learning with diverse black box predictors in maximising ensemble rewards as the proxy of model performance potential while maintaining an appropriate proportionate allocation and the independent and identically distributed (i.i.d.) assumption. A comprehensive evaluation of the RDS procedure is conducted on four real-world datasets, including Madelon, Drug Reviews, MNIST, and Kalapa Credit Scoring Challenge, with coverage of machine learning tasks such as binary classification, multi-class classification, and regression on multivariate, textual, and visual data. The experimental results evidently demonstrate consistent performance improvements of trainable data samples over classical or prior data selection. Hence, we advocate the use of RDS for data splitting in the early stage of machine learning tasks for parameter tuning, model selection and overfitting prevention, as well as, sampling in large-scale AI competitions for searching the best possible and shift-stable solutions.

1. Introduction

The symbiotic relationship between data characteristics and parametric models is essential for developing effective and reliable artificial intelligence (AI) systems. Mismatches in data distributions for model development may lead to detrimental performance, thereby impairing the fit of otherwise state-of-the-art models in real-world applications [Szegedy et al. \(2014\)](#). Proper data management and sampling techniques, thus, are crucial to the reliability of machine learning tasks in the search for the best possible solutions and parameters to generalise new observations adequately [Zhou et al. \(2018\)](#). In common scenarios, the available data are divided into numerous parts using randomization, even with multi-

ple random seeds, for model selection and evaluation, which are susceptible to detrimental issues on learning performance, including concept drift and evidence ambiguity [Bach and Maloof \(2010\)](#). The model concept, or mappings from data observations to outputs, may drift rendering sub-optimal fit with new data points due to hidden context changes or data-related problems. Often, there is simply insufficient or inappropriate information presented in the data samples to support adequate predictive strength, which is also known as evidence ambiguity [Lee et al. \(2016\)](#). Hence, the nexus between data points and model outputs plays a crucial role in data selection to improve generalisation and to mitigate such issues [Zhou et al. \(2018\)](#).

The process of selecting subsets of data points for training, selection and evaluation is a fundamental step in various machine learning tasks. We propose a Reliable-Data-Split (RDS) procedure to learn how to split data effectively predicated on base learners in searching for useful models and insights. Moving beyond classical data splitting techniques, based on stratification and randomization (train-test-split procedures), our procedure employs a reinforced selection strategy to sample data subsets using deep reinforcement learning to maintain an appropriate proportionate allocation and the independent and identically distributed (i.i.d.) assumption. Inspired by the Black Box Shift Estimation (BBSE) technique [Lipton et al. \(2018\)](#), we introduce the ensemble rewards, collectively soft voting and stochastic choice, with the use of multiple black box predictors to enhance the representational ability of the data and to select subsets of data points according to future model potential [Gong et al. \(2019\)](#). The utilization of black box predictors with invertible confusion matrices, even when biased, uncalibrated, and inaccurate, has been empirically studied as an effective way to handle dataset shifts [Lipton et al. \(2018\)](#); [Rabanser et al. \(2019\)](#).

The paper provides a comprehensive evaluation of the RDS procedure using four real-world datasets, including NIPS 2003 Feature Selection Challenge - Madelon, Kaggle Hackaton - Drug Reviews, MNIST, and Kalapa Credit Scoring Challenge. Our experiments cover a range of machine learning tasks such as binary classification, multi-class classification, and regression on multivariate, textual, and visual data. The results evidently highlight consistent performance improvements of trainable data samples over classical or prior data selection. Therefore, we advocate the use of reliable data split to maximize model potential based on diverse black box predictors while maintaining distributional properties in the early stage of machine learning tasks, when adequate training and validations are required for parameter tuning, model selection and overfitting prevention. Furthermore, with the rise of AI competitions, we recommend RDS to split public benchmarking data subsets, including train, validation and test sets for best possible and shift-stable solutions. This might be helpful to mitigate dataset shift risks and unwanted impacts in large-scale projects or machine learning competitions, such as Santander Value Prediction Challenge and VSB Power Line Fault Detection. In this paper, the publicly available state-of-the-art algorithms for the four datasets are reproduced for model selection and performance evaluation to demonstrate the usefulness of RDS.

The paper has the following contributions.

- We introduce Reliable Data Split (RDS), a method to approximate optimum data sampling for maintaining distributional properties and using ensemble black box estimators to maximise the performance potential. Two types of ensemble mechanisms,

including soft voting and stochastic choice, are investigated for better representational ability in the search for reliable learning.

- We implement an end-to-end reinforced selection strategy for trainable data sampling with configurable properties of data distributions, including sampling ratio, i.i.d., and reference distribution of the data generation process.
- Comprehensive experiments are conducted to compare RDS against other data splitting methods on four real-world datasets. The results highlight consistent performance improvements with the use of base models and public available solutions.

2. Related Work

In machine learning, generalisation, or the ability to adapt new, previously unseen observations, plays a vital role in creating useful models [Cohn et al. \(1994\)](#). It entails the process of data sampling, which is employed to select and manipulate a representative subset of data points for performance estimation. Early approaches, such as simple random sampling or stratified sampling, have been widely adopted in numerous machine learning tasks to date. The use of simple randomisation (e.g., Knuth’s algorithm [Knuth \(1997\)](#)) in data selection is overly popular; however, it is susceptible to many sampling issues such as violation of statistical independence, bias or covariate shift [May et al. \(2010\)](#); [Bach and Maloof \(2010\)](#). Stratification technique is used to partition the dataset into homogenous strata to ensure the adequate representation of data points [Cochran \(2007\)](#). Overall, such sampling approaches tend to suffer from covariate shift or/and label shift. Covariate shift or *sample selection bias* is well-studied with many approaches [Zadrozny \(2004\)](#); [Huang et al. \(2007\)](#); [Sugiyama et al. \(2008\)](#) trying to tackle the shift, by correcting models via weighting examples in ERM by $q(x)/p(x)$ [Shimodaira \(2000\)](#), or by re-weighting training points using Kernel Mean Matching (KMM) [Gretton et al. \(2009\)](#). In turn, [Storkey \(2009\)](#) studies label shift with unknown $q(y)$ and proposes placing a prior over $p(y)$ and inferring the posterior distribution from unlabelled test data, where [Chan and Ng \(2005\)](#) use EM algorithm to infer the label distribution $q(y)$. KMM can also be extended to study the label shift problem [Zhang et al. \(2013\)](#). [Lipton et al. \(2018\)](#) explore the idea of using black box predictor to detect the shift in the label distribution between source and target sets. [Rabanser et al. \(2019\)](#) unify multiple methods to detect dataset shift including both covariate shift as well as label shift.

In computational learning theory, model performance and complexity have been formalised as factors to generalisation bounds according to Occam’s razor [Rasmussen and Ghahramani \(2001\)](#). Hold-out method [Schorfheide and Wolpin \(2012\)](#) for data selection is commonly performed to estimate the predictive performance of a learner, which can be repeated multiple times to improve stability with less variance. Furthermore, modern datasets are typically associated with heterogeneous features, ambiguous evidence, and complex dependencies, thereby leading to concept drift in model performance [Minku et al. \(2009\)](#). Importance sampling by re-weighting data points has been explored as a remedial mechanism [Sugiyama et al. \(2008\)](#); [Mahmood et al. \(2014\)](#). In recent years, many researchers have approached model drift and related dataset issues with ensemble learning [Bach and Maloof \(2010\)](#); [Minku et al. \(2009\)](#); [Kolter and Maloof \(2007\)](#); [Bach and Maloof \(2008\)](#). Multiple

base models can be trained on blocks of data samples to address uncertainty by injecting model diversity, in the hope of maximising performance generalisation [Lakshminarayanan et al. \(2017\)](#). With recent advances in reinforcement learning [Peng et al. \(2019\)](#); [Buckman et al. \(2018\)](#); [Lee et al. \(2016\)](#), we explore how to sample informative data points that best generalise machine learning models with ensemble learning and model diversification.

3. Proposed Framework

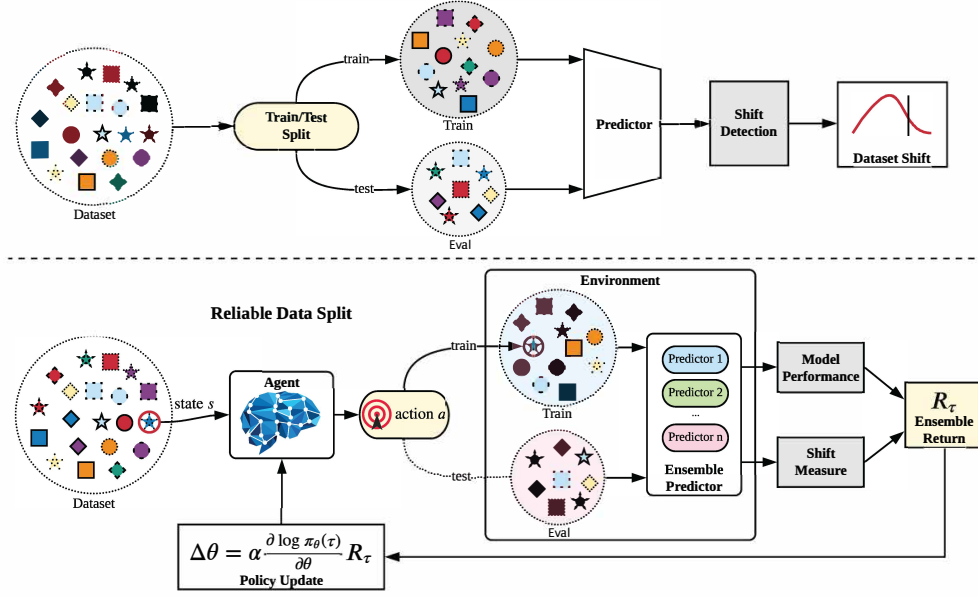


Figure 1: Reliable Data Split (RDS) procedure for maximizing the ensemble rewards of multiple black box predictors as the proxy of model potential while maintaining distributional properties (e.g., the i.i.d. assumption). In each episode, a set of samples are selected to belong to training or evaluation sets, to be used by an ensemble model and a shift evaluator for computing rewards. The learning process stopped when the stopping conditions are met (e.g., after a number of iterations or when the sampling criteria are achieved).

The ultimate goal is to maximize model performance potential, in which one solution is to introduce priors into the data sampling process. Our proposed approach shares the same principle of reducing shifts yet comprises a sophisticated method for sampling. The method is able of splitting one dataset into multiple subsets that maintain several desired properties. It consists of representative and diverse data patterns, therefore, maximising candidate learning potentials as well as addressing dataset shift problem in machine learning. Ultimately, we will obtain data subsets (e.g., training and evaluation) which are capable of searching for models achieving good shift-stable learning performance.

3.1. Problem Formulation

Let $\mathfrak{D} = \{(x_i, y_i) | x_i \in X, y_i \in Y\}_{i=1}^M$ denote a dataset of size M , where X are inputs and Y are dependent outputs. There are two objectives that we would want to achieve, (1) to maximise learning potentials of model candidates, (2) to avoid dataset shift which is critical for model generalisation in machine learning. To that end, we propose a data sampler $\omega : \{X, Y\} \rightarrow \varsigma(\{X, Y\})$ generating multiple subsets $\mathfrak{d}$ of the dataset with several properties.

Firstly, an optimal data sampler ω^* should maximise the ensemble learning performance of multiple base predictors as the proxy of model potential thus maintaining diversity of data patterns. There are advantages of using ensemble learning as compared to single black-box method [Rabanser et al. \(2019\)](#), (1) improving representational abilities, and (2) enriching informative patterns lying within the data (i.e., pattern diversity). Consider a machine learning task such as classification or regression, in which it aims at modeling $f : x \rightarrow y$ using training data samples in light of maximising its task performance. We advocate the use of diverse K learners $\mathcal{F} = \{f_1, f_2, \dots, f_K\}$, including stable methods (e.g., support vector machines or regularised least square regression) and less stable methods (e.g., neural networks). We seek to find ω^* such that:

$$\omega^*(\{X, Y\}) :=_{\varsigma(\{X, Y\})} \mathfrak{E}(f_{\epsilon}(\varsigma(\{X, Y\}))) \quad (1)$$

where f_{ϵ} is an ensemble predictor of $\mathcal{F}$ and $\mathfrak{E}$ is the criterion that measures the performance of f_{ϵ} .

Secondly, we posit that the sampling procedure is stochastic, in which the allocation of samples to subsets is based on parametric probability distributions p with the parameters θ .

$$\varsigma(\{x, y\}) \sim p^{\theta} \quad (2)$$

Thirdly, the sampler should maintain a desired sampling ratio τ , thus,

$$\mu(p^{\theta}) \approx \tau \quad (3)$$

where μ is the mean of the probability distributions p^{θ} .

Fourthly, data samples are assumed to be independent and identically distributed (i.i.d). Hence, data subsets are representative of the true population in respect to statistical independence. For each subset $\mathfrak{d}$ of size $M_{\mathfrak{d}}$,

$$(\{x_i, y_i\}) \sim \mathbb{P}(X, Y) \quad \forall i = 1, \dots, M_{\mathfrak{d}} \quad (4)$$

where $\mathbb{P}$ is the true distribution of the dataset. In this framework, we advocate the use of black box predictors to prevent distribution label shift [Lipton et al. \(2018\)](#). It is also empirically shown that black box predictors are able of detecting a broad set of shifts [Rabanser et al. \(2019\)](#), therefore, allowing us to reduce the general forms of nonstationarity such as label shift and concept drift across the subsets.

$$(\{x^*, f_{\epsilon}(x^*)\}) \sim \mathbb{P}(X, Y) \quad (5)$$

where x^* is unlabelled evaluation data.

With the four properties being described, we aim at searching for optimal ω^* yielding shift-stable solutions; nevertheless, it is NP-Hard with the possibility of solving with approximation. Therefore, we propose a reinforcement learning approach to discovering how to sample by approximating the shift-stable solutions.

3.2. Reliable Data Split (RDS)

We propose Reliable Data Split (RDS) framework to maximise model learning performance. RDS is based on reinforcement learning and can be defined under the Markov Decision Process (MDP). Subsequently, we will use the language of reinforcement learning to describe the proposed framework.

We posit the use of a data sampler ω to create two subsets, $\mathfrak{d}_{train} = \{X_{train}, Y_{train}\}$ and $\mathfrak{d}_{eval} = \{X_{eval}, Y_{eval}\}$, to discuss our approach without loss of generality.

In RDS, the agent (data sampler) employs a reinforced selection strategy to interact with an environment. As illustrated in Figure 1, at each step, the agent receives a data sample (x, y) (state s), classifies which subset the sample belongs to (action a) until it reaches the final state completing a sampling process (one episode). It then receives a reward r , ensemble model performance, given by the environment based on the outcome of the action sequence. The agent reaches an optimum goal through its interactions with the environment to accumulate maximum possible rewards. It can be described with a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T})$ as follows:

- $\mathcal{S}$: finite set of states, where the decision process is evolved for every data point.
- $\mathcal{A}(s) = \{j_{train}, j_{eval}\}$: discrete set of agent's actions.
- $\mathcal{R}$: ensemble return for agent's sequence of actions.
- $\mathcal{T}$: agent's transition from the current state to the next state.

Policy Optimisation. The framework employs a stochastic policy π which defines the probability of performing action a by the agent given the state s ; thus $\pi^\theta(a|s) \approx p^\theta(a|s)$ where the probability distribution is determined by the parameter θ according to Eq(2). Given policy π , RDS starts from observing an initial state s_0 according to the probability distribution p_0 . At each step of interaction t , it evolves according to:

$$s_{t+1} = \mathcal{T}(s_t, a_t \sim \pi^\theta(a|s_t)) \quad (6)$$

We denote τ as the trajectory of the RDS, where $\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$. The transition $\mathcal{T}$ is deterministic as the agent moves from the current state s_t to the next state s_{t+1} according to the order of observations in the dataset. The optimisation problem in RDS is expressed by finding a good set of parameters θ to maximise the expected return:

$$\theta^* =_\theta E_\tau[R_\tau | \pi^\theta] \quad (7)$$

where R_τ is the finite-horizon undiscounted return computed based on the trajectory of T steps.

Algorithm 1 Training Algorithm for RDS^{DET}

Require: dataset $\mathcal{D} = \{(x_t, y_t) | x_t \in X, y_t \in Y\}_{t=1}^T$, ensemble learner f_ϵ , sampling ratio $\mathfrak{r}$

- 1: **procedure** RDS($\mathcal{D}, f_\epsilon, \mathfrak{r}$)
- 2: **initialise** $\pi^\theta \leftarrow \text{pretrain}(\mathfrak{r})$ **forall** $i \leftarrow 1, L$ **do**
- end**
- L : number of epochs
- 3: $V \leftarrow \emptyset$ $\triangleright V$: replay memory **for** $t \leftarrow 1, T$ **do**
- 4: **end**
- $s_t \leftarrow (x_t, y_t)$
- 5: **select** action $a_t \sim \pi^\theta(a | s_t)$
- 6: $V \leftarrow V \cup \{(s_t, a_t)\}$ $\triangleright$ Save to V
- 7:
- 8: $\mathfrak{d}_{train} \leftarrow \{s \in V | a = j_{train}\}$
- 9: **train** $f_\epsilon^{DET}(\cdot)$ on the sampled training set $\mathfrak{d}_{train}$
- 10: $\mathfrak{d}_{eval} \leftarrow \{s \in V | a = j_{eval}\}$
- 11: **compute** $R_\tau = \mathfrak{G}(\mathfrak{d}_{eval}, f_\epsilon^{DET}(\mathfrak{d}_{eval}))$ $\triangleright$ Eq(11)
- 12: **update** the policy π^θ
- 13:
- 14: **generate** $\omega(\{X, Y\})$ $\triangleright$ Eq(10)
- 15: $\mathfrak{d}_{train}, \mathfrak{d}_{eval} \leftarrow \omega(\{X, Y\})$
- 16: **return** $(\mathfrak{d}_{train}, \mathfrak{d}_{eval})$
- 17: **end procedure**

In RDS, we investigate the use of supervised estimators, which mimic the input-output process in nature. These function approximation methods may range from linear functions to decision trees or artificial neural networks. They receive data samples as observations of the state to predict values, where $f : s \rightarrow \hat{v}(s)$. In general, the objective function for machine learning is specified as $\mathfrak{G}(s, \hat{v}(s)) \in \mathbb{R}$. At the end of each episode, we apply our function approximation on the training dataset $\mathfrak{d}_{train} = \{s \in \mathcal{S} | a = j_{train}\}$ and the evaluation dataset $\mathfrak{d}_{eval} = \{s \in \mathcal{S} | a = j_{eval}\}$. Once the transition is terminated at step T , we compute:

$$R_\tau = \mathfrak{G}(\{s|a\}, \hat{v}(\{s|a\})) \quad (8)$$

We utilise the policy gradient method to address the optimisation problem, in which policy weights are updated by the stochastic gradient optimisation at the end of every episode as the following:

$$\Delta\theta \approx \frac{\partial \log \pi^\theta(\tau)}{\partial \theta} R_\tau \quad (9)$$

where $\pi^\theta(\tau) = \prod_{t=0}^T \pi^\theta(a_t | s_t)$ is the trajectory probability of the action sequence τ .

Once the policy has converged, we arrive at a good approximation of our data sampler ω in Eq(1) as the following:

$$\omega(\{X, Y\}) \approx \{a_1^*, a_2^*, \dots, a_T^*\} \quad (10)$$

where $a_t^* =_{a \in \mathcal{A}(s_t)} \pi^{\theta^*}(a|s_t)$.

In general, the RDS process starts with sampling a training subset $\mathfrak{D}_{train}$ and a evaluation subset $\mathfrak{D}_{eval}$ of the dataset $\mathfrak{D}$ according to the policy π . We apply the function approximation f by training on the training set $\mathfrak{D}_{train}$ and evaluating on the eval set $\mathfrak{D}_{eval}$ to obtain an expected return R_τ . The policy π is then updated accordingly and a new episode is started to reach the convergence of π . The training algorithms for deterministic variant RDS^{DET} and stochastic variant RDS^{STO} are presented in Alg.1 and Alg.2, respectively.

Convergence. We posit that the learning convergence of RDS is inherited from the convergence of the policy gradient method [Sutton et al. \(2000\)](#). The predictors composing reward mechanism are designed with consistent parameter initialisation and hyper-parameters; hence, the reward values are consistent for the same sampled outcome of the dataset.

Complexity. The computational complexity of RDS is $\mathcal{O}(N(MD + CK))$, where N is the number of episodes for policy updating, M is the size of the dataset, D is the cost for state updating, and CK is the computational cost for function approximation from K predictors.

3.3. Reward Mechanisms

Our target is to train an agent to draw relevant samples with the policy π to maximise the expected return $E_\pi[R]$, which entails performance potentials of value functions. In this paper, we consider wide-ranging value functions such as linear estimators, decision-trees, and neural networks, which are commonly used in discriminative modelling. Let f be any arbitrary function learners, then the RDS process converges with the specified rewards [Sutton et al. \(2000\)](#).

We design the learning environment with the use of ensemble method of multiple function approximators to emphasize representative and diverse patterns for data selection by design; because each base model provides outcome reflecting multi-modal belief [Gong et al. \(2019\)](#). Let f_ϵ denote the ensemble function approximator; hence we have $f_\epsilon = \Xi(f_1, f_2, \dots, f_K)$, where K is the number of base learning models for evaluating performance metrics. We fix the training procedure of the supervised learners, including parameter initialisation, model architecture, hyper-parameters, and random seeds to ensure the same output from a given state for reproducibility. This paper investigates several reward mechanisms, including soft voting and stochastic choice, which are applicable for both classification and regression problems. In the soft voting approach, we define RDS^{DET} using an ensemble approximator $f_\epsilon^{det} : s \rightarrow \bar{v}(s)$ with the following value function:

$$\bar{v}(s) = \frac{1}{K} \sum_{k=1}^K \hat{v}_k(s) \quad (11)$$

Thus, the environment is observed as deterministic with the reward $R_\tau^{\text{perf}} = \mathfrak{G}(\{s|a\}, \bar{v}(\{s|a\}))$.

In addition, we define a stochastic RDS^{STO} process $f_\epsilon^{sto} : \{f_k, k = 1, 2, \dots, K\}$ depends on the base models randomly picked from a stationary distribution ρ at each epoch. It is desirable that this stochastic behaviour may overcome local optimisation despite the noise introduced. We define:

$$\check{v}(s) = \hat{v}_k(s); k \sim \rho \quad (12)$$

Algorithm 2 Training Algorithm for RDS^{STO}

Require: dataset $\mathcal{D} = \{(x_t, y_t) | x_t \in X, y_t \in Y\}_{t=1}^T$, ensemble learner f_ϵ , sampling ratio $\mathfrak{r}$

```

1: procedure RDS( $\mathcal{D}, f_\epsilon, \mathfrak{r}$ )
2:   initialise  $\pi^\theta \leftarrow \text{pretrain}(\mathfrak{r})$  forall  $i \leftarrow 1, L$  do
      end
       $L$ : number of epochs
3:    $V \leftarrow \emptyset$   $\triangleright V$ : replay memory for  $t \leftarrow 1, T$  do
4:   end
       $s_t \leftarrow (x_t, y_t)$ 
5:   select action  $a_t \sim \pi^\theta(a | s_t)$ 
6:    $V \leftarrow V \cup \{(s_t, a_t)\}$   $\triangleright$  Save to  $V$ 
7:
8:    $\mathfrak{d}_{train} \leftarrow \{s \in V | a = j_{train}\}$ 
9:   choose  $f_\epsilon^{STO}(\cdot)$  from stationary distribution  $\rho$ 
10:  train  $f_\epsilon^{STO}(\cdot)$  on the sampled training set  $\mathfrak{d}_{train}$ 
11:   $\mathfrak{d}_{eval} \leftarrow \{s \in V | a = j_{eval}\}$ 
12:  compute  $R_\tau = \mathfrak{G}(\mathfrak{d}_{eval}, f_\epsilon^{STO}(\mathfrak{d}_{eval}))$   $\triangleright$  Eq(12)
13:  update the policy  $\pi^\theta$ 
14:
15:  generate  $\omega(\{X, Y\})$   $\triangleright$  Eq(10)
16:   $\mathfrak{d}_{train}, \mathfrak{d}_{eval} \leftarrow \omega(\{X, Y\})$ 
17:  return  $(\mathfrak{d}_{train}, \mathfrak{d}_{eval})$ 
18: end procedure

```

The environment, therefore, is observed as stochastic with the reward $R_\tau^{\text{perf}} = \mathfrak{G}(\{s|a\}, \check{v}(\{s|a\}))$.

We argue that the choice of base models is crucial to achieving higher learning potentials with model diversity. In addition, pre-processing steps or pre-trained feature mappings can be adopted in these learners to provide better representational abilities.

Furthermore, we implement shift evaluation metrics on the search for shift-stable learning. A reward mechanism is designed to ensure that testing predictions are similar to the true distribution as described in Eq(5). We adopt the Chi-Square test of independence, aligned with previous studies Rabanser et al. (2019); Lipton et al. (2018). Hence, the shift-avoidance reward is defined as $R_\tau^{\text{shft}} = \chi_p^2(f_\epsilon(s), \mathbb{P})$.

The ensemble return R_τ is computed as the following:

$$R_\tau = \lambda_p * R_\tau^{\text{perf}} + \lambda_s * R_\tau^{\text{shft}} \quad (13)$$

where λ_p and λ_s are weight factors for performance-based and shift-based rewards respectively.

3.4. Policy Learning

We implement the policy learning using the Gated Recurrent Unit Bahdanau et al. (2014) with the feature size of the dataset $\mathcal{D}$. It is an intuitive choice as the gated networks support

the data selection based on the sequence of samples similar to the desired agent’s brain. In the learning, the state s_t at the step t is encoded to create a hidden vector presentation h_t of s_t . With two reset and update gates, the computation of h_{t+1} is described as the following:

$$h_{t+1} = \begin{cases} GRU(h_t, x_{t+1} \oplus y_{t+1}) & \text{if } (x_t, y_t) \text{ is selected} \\ GRU(h_{t-1}, x_{t+1} \oplus y_{t+1}) & \text{otherwise} \end{cases} \quad (14)$$

A linear layer is adopted to derive the probability distribution of the action $p^\theta(a|s)$. Moreover, the policy π^{θ_0} is pre-trained based on the sampling ratio τ to achieve faster convergence.

We use the log of the action probability an equivalent loss function with the learning factor α :

$$\mathcal{L}^\theta = \alpha \frac{-\partial \log \pi^\theta(\tau)}{\partial \theta} R_\tau \quad (15)$$

Distributional Regularisation. We implement a regularisation loss to ensure the sampling ratio based on the distributional property of the action probability in Eq(3) as the following:

$$\mathcal{L}^{r-reg} = \gamma L1(\mu(\pi^\theta(\tau)), \tau) \quad (16)$$

where γ is the scale factor and τ is the sampling ratio.

In addition, we design a regularisation mechanism to ensure that training samples and evaluation samples are drawn from the same distribution as described in Eq(4). This is important for both classification (e.g., class ratios) and regression (e.g., identically distributed). Given probability density distributions of the training set p_{train} and the evaluation set p_{eval} . We define:

$$\mathcal{L}^{i-reg} = \psi KL(p_{eval}, p_{train}) \quad (17)$$

where ψ is the scale factor and KL is the Kullback–Leibler divergence.

The final loss for our policy optimisation is computed as:

$$\mathcal{L} = \mathcal{L}^\theta + \mathcal{L}^{r-reg} + \mathcal{L}^{i-reg} \quad (18)$$

4. Experiments

We conduct experiments to examine the effectiveness of the RDS method in well known public benchmark datasets. It is done via evaluating *shift-stable learning* reflected by the performance evaluated on proposed data samples by RDS in comparison with other techniques (e.g., stratified).

4.1. Experimental Setup

Datasets. Four real-world datasets are selected for the analysis of the effectiveness of RDS. Our study simulates the way that the datasets are prepared for the tasks, given no prior knowledge on existing or public solutions. The datasets are detailed as follows.

- **Madelon (MDL)** Guyon et al. (2005) was artificially developed for the NIPS 2003 feature selection challenge. It has 500 numerical features, in which 20 real features and 480

distractors have no predictive capacity. Several pre-processing techniques were adopted to conceal the origin and patterns of the dataset on the search for functional feature extractors. We employ bare-bone Logistic Regression (LR), Random Forest (RF), and Multi-Layer Perception (MLP) in our experiments. And a pipeline of stability selection and Logistic Regression of feature interactions is adopted for public benchmarking [Cannon \(2018\)](#).

- **Drug Review (DR)** [Gräßer et al. \(2018\)](#) provides patients’ reviews on specific drugs crawled from multiple online pharmaceutical review sites. It contains categorical features including drug name and patient condition, review text and date, and numerical features including review rating and useful counts. In total, there are 215,063 examples, which is split into a training set of 75% and a evaluation set of 25%. In this experiment, we use three base learners including Ridge Regression (Ridge), Multi-Layer Perception (MLP), and Convolutional Neural Network (CNN).

- **MNIST** [LeCun \(1998\)](#) consists of 70,000 hand-written digits and is one of the most well-known datasets in the deep learning community. MNIST is selected for experiments since it represents very well for multi-class classification task on images. MNIST considered a balanced image classification data, and it is divided into 60K samples for training and 10K samples for evaluation. In this experiment, we use three base learners including Logistic Regression (LR), Random Forest (RF), and Convolutional Neural Network (CNN).

- **Kalapa Credit Scoring (KLP)** [Kalapa \(2020\)](#) is a data challenge for credit scoring task. The dataset consists of 30,000 training and 20,000 evaluation examples with binary labels (GOOD and BAD). Each example is associated with 62 variables, including demographics and financial status. There is an imbalance on the label distribution with a ratio of 1.6% (i.e., only 486 BAD examples among 30,000 in training). Moreover, 40 data fields have missing values of more than 30%, which increases the difficulty in finding a good data selection for the data challenge. We consider three models namely Logistic Regression (LR), Random Forest (RF), and Multi-Layer Perception (MLP) as the base learners. The first ranked solution [Le \(2020\)](#) is selected as the public model for comparisons.

Implementation. The source code of RDS is implemented in Pytorch whilst learning environments are built flexibly by various learning frameworks such as Keras, Tensorflow or Scikit-learn. Environmental learning models are optimised concurrently using a common evaluation metric. For the policy optimisation, the number of hidden unit of GRU is 256. The learning is run on 3-400 episodes with the RMSprop optimiser and the initial learning rate of 0.001. Scaling factors (α, γ, ω) are empirically selected, i.e., (1.0, 0.9, 0.1) for Madelon, (1.0, 1.0, 40) for Kalapa, (1.0, 0.1, 0.01) for MNIST and (1.0, 0.9, 0.1) for Drug Review. For KLP and DR datasets, we employ FastText [Bojanowski et al. \(2016\)](#) and BERT [Devlin et al. \(2019\)](#) language models for extracting representation for textual contents. All experiments are conducted on a similar computational environment of Intel(R) Xeon(R) Gold 6244 CPU @ 3.60GHz, 256GB Ram, and a Titan RTX 2080Ti GPU card.

Pre-processing. For Madelon, several pre-processing techniques were previously adopted to conceal the origin and patterns of the dataset on the search for functional feature extractors. For Drug Review, Ridge and MLP use average-pooling word embeddings from BERT-Base model of 768 dimensions, while CNN word embeddings are initialised from

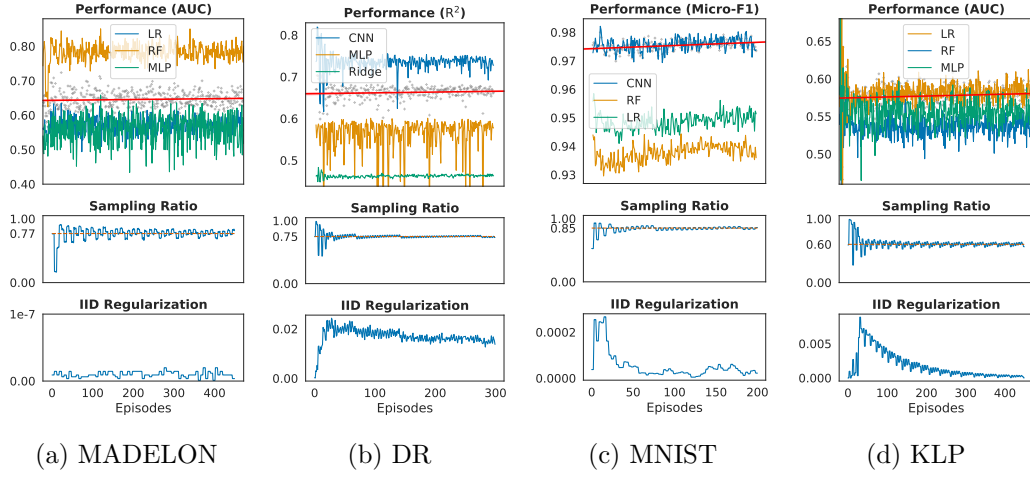


Figure 2: Learning Dynamics for Deterministic Soft-Voting Reward Mechanism

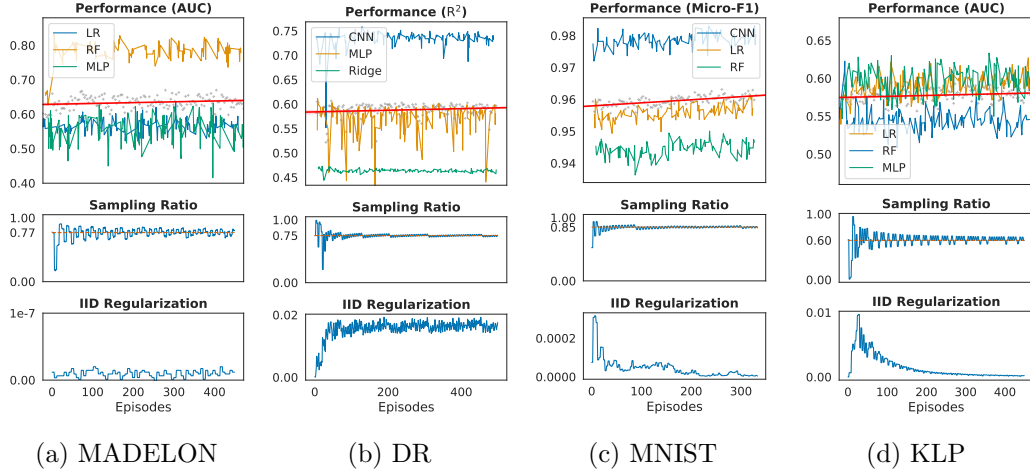


Figure 3: Learning Dynamics for Stochastic Choice Reward Mechanism.

pre-trained word2vec of 300 dimensions.. For MNIST, we extract Histogram of Oriented Gradients (HOG) features for policy learner, LR, and RF algorithms. To reduce the dimension, PCA is applied with the `n_components` of 0.95. It means the number of components is selected such that the amount of variance that needs to be explained is greater than the specified percentage (i.e., 95%). CNN is run directly on RGB codes with normalisation. For Kalapa, we employ a fine-tuned FastText [Bojanowski et al. \(2016\)](#) model of 32 dimensions to extract the representation for textual contents including *'Province'*, *'District'*, *'Job'*.

Table 1: Madelon Experiment. *Public* denotes a public solution of Cannon (2018).

Sampling	#Sample		Class Ratio		LR	RF	MLP	Ensemble	Public
	Train	Eval	Train	Eval					
Preset	2000	600	1.0000	1.0000	.6019	.8106	.5590	.6783	.9063
Random	2000	600	.9920	1.0270	.5742	.7729	.5774	.6453	.9002
Stratified	2000	600	1.0000	1.0000	.5673	.7470	.6153	.6360	.8828
RDS ^{DET}	2001	599	1.0375	.9137	.6192	.8050	.6228	.6973	.8915
RDS ^{STO}	2021	579	1.0010	.9966	.6192	.8050	.6050	.6947	.9106

Table 2: Drug Review Experiment. *Public* denotes a public solution, Bi-LSTM with Attention, on Kaggle.

Sampling	Train	Eval	Ridge	MLP	CNN	Ensemble	Public
Preset	161,297	53,766	.4580	.5787	.7282	.6660	.7637
Random	161,297	53,766	.4597	.4179	.7353	.6485	.7503
RDS ^{DET}	162,070	52,993	.4646	.5776	.7355	.6692	.7649
RDS ^{STO}	161,944	53,119	.4647	.5370	.7509	.6562	.7600

Other fields are applied with traditional feature engineering techniques, e.g., MinMaxScaler, OrdinaryEncoder, dummy variables.

Baselines. We compare our proposed RDS approach with several traditional data sampling methods, including simple randomisation denoted as *Random*, stratification (only for classification) denoted as *Stratified*. We also include comparisons with available splitting denoted as *Preset*, which is provided either by the organisers of competitions or the authors of the datasets. Moreover, we select a number of prominent solutions which have been shared by the members of the public to examine the effects of various techniques on the datasets.

Metrics. For Madelon and Kalapa datasets, the tasks are binary classification; therefore, we use AUC to measure model performance. In turn, we employ Micro-F1 metric for the task of multi-class classification on MNIST dataset. For experiment on Drug Review, we use R-squared (R^2) to measure performance of the models as the task is regression.

4.2. Results and Discussion

The results demonstrate that our proposed RDS approach with various reward mechanisms works steadily with the four datasets. Figure 2 depicts the learning dynamics of RDS^{DET}, in which the regression line is highlighted in red to indicate the improvements over time of the designed agent with diversification of multiple base models. Given a finite number of episodes, RDS^{DET} establishes desirable optimisation behaviours regularised by data property assumptions. Likewise, Figure 3 illustrates the learning dynamics of stochastic reward mechanism, in which lesser numbers of approximations are exhibited in the performance

Table 3: MNIST Experiment. *Public* is a solution based on CNN architecture. It is noted that the public solution is not the same CNN’s architecture used in RDS, which has fewer layers and takes shorter time to train.

Sampling	#Sample		Class Ratio		LR	RF	CNN	Ensemble	Public
	Train	Eval	Train	Eval					
Preset	60000	10000	.8571	.1429	.9647	.9524	.9824	.9819	.9917
Random	59500	10500	.8500	.1500	.9603	.9465	.9779	.9768	.9914
Stratified	59500	10500	.8500	.1500	.9625	.9510	.9795	.9792	.9901
RDS ^{DET}	59938	10062	.8562	.1438	.9495	.9382	.9757	.9769	.9927
RDS ^{STO}	59496	10504	.8499	.1501	.9583	.9486	.9851	.9830	.9931

Table 4: Kalapa Experiment. *Public* denotes the first solution from Kalapa challenge on the private leader board.

Sampling	#Sample		Class Ratio		LR	RF	MLP	Ensemble	Public
	Train	Eval	Train	Eval					
Preset	30000	20000	.0165	.0186	.5799	.5517	.5635	.5723	.5953
Random	30000	20000	.0169	.0179	.5886	.5374	.5914	.5856	.6042
Stratified	30000	20000	.0173	.0173	.5952	.5608	.5780	.5983	.6014
RDS ^{DET}	29999	20001	.0180	.0163	.6045	.5350	.5802	.6057	.5362
RDS ^{STO}	30031	19969	.0172	.0174	.5997	.5491	.6354	.6072	.6096

of all datasets. The results show better optimisation of the learning gradients with this efficient method.

In details, RDS^{DET} yields good performance for the ensemble performance which has been directly optimised for. This upward trend can be clearly observed across all datasets. RDS^{STO} demonstrates clear outperformance for the base learners, especially the results are significant for LR model on *Madelon* (Table 1), CNN models on *DR* (Table 2) and *MNIST* (Table 3), as well as *KLP* (Table 4). Amongst the baselines, *Stratified* has the strength of maintaining class ratios for the task of classification, which can also be maintained by the proposed RDS methods. The *Preset* splitting, given by the competition organiser or authors, appears to be either *Random* or *Stratified*. Thus, they obtain comparable performance to randomisation and stratification but worse than RDS variants. Although the preset allocation performs well in some settings, the adequate performance of the RDS is consistently observed in both ensemble evaluation and public benchmarking. The stochastic choice mechanism gains some advantages over the previously designed algorithms. Moreover, the assumption of statistical independence holds a critical impact on the learning of the agent, which must be considerably regularised for imbalance datasets. Trainable data split using reinforced selection strategy achieves consistently good performance based on ensemble learning and publicly available solutions; thus, higher learning potentials are yet to be explored.

5. Conclusion

This paper proposes Reliable-Data-Split (RDS) procedure, which learns to select representative samples. The objective is to emphasise ensemble learning performance of the learners, by maximising learning potentials of ensemble predictors, preventing dataset shift, as well as maintaining distributional properties of the split subsets. We introduce different reward mechanisms including soft voting and stochastic choice to train optimal sampling policy under reinforcement learning framework. Experiments conducted on four real-world datasets including MADELON, Drug Review, MNIST and Kalapa evidently highlight the benefits of using RDS over classical sampling approaches. Moreover, RDS’s sampling approach is configurable and can be applied to many different types of data and models. Code and datasets used in our experiments, which have been used in multiple international competitions (anonymized), will be made available upon publication.

References

- Stephen Bach and Mark Maloof. A bayesian approach to concept drift. In *NeurIPS*, pages 127–135, 2010.
- Stephen H Bach and Marcus A Maloof. Paired learners for concept drift. In *ICDM*, pages 23–32, 2008.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *arXiv preprint arXiv:1607.04606*, 2016.
- Jacob Buckman, Danijar Hafner, George Tucker, Eugene Brevdo, and Honglak Lee. Sample-efficient reinforcement learning with stochastic ensemble value expansion. In *NeurIPS*, pages 8224–8234, 2018.
- Blake Cannon. Madelon madness feature selection. <https://github.com/wblakecannon/madelon-madness>, 2018.
- Yee Seng Chan and Hwee Tou Ng. Word sense disambiguation with distribution estimation. In *IJCAI*, volume 5, pages 1010–5. Citeseer, 2005.
- William G Cochran. *Sampling techniques*. John Wiley & Sons, 2007.
- David Cohn, Les Atlas, and Richard Ladner. Improving generalization with active learning. *Machine learning*, 15(2):201–221, 1994.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, pages 4171–4186, 2019.
- Zhiqiang Gong, Ping Zhong, and Weidong Hu. Diversity in machine learning. *IEEE Access*, 7:64323–64350, 2019.

- Felix Gräßer, Surya Kallumadi, Hagen Malberg, and Sebastian Zaunseder. Aspect-based sentiment analysis of drug reviews applying cross-domain and cross-data learning. In *DH*, pages 121–125, 2018.
- Arthur Gretton, Alex Smola, Jiayuan Huang, Marcel Schmittfull, Karsten Borgwardt, and Bernhard Schölkopf. Covariate shift by kernel mean matching. *Dataset shift in machine learning*, 3(4):5, 2009.
- Isabelle Guyon, Steve Gunn, Asa Ben-Hur, and Gideon Dror. Result analysis of the nips 2003 feature selection challenge. In *NeurIPS*, pages 545–552, 2005.
- Jiayuan Huang, Arthur Gretton, Karsten Borgwardt, Bernhard Schölkopf, and Alex J Smola. Correcting sample selection bias by unlabeled data. In *NeurIPS*, pages 601–608, 2007.
- Kalapa. Kalapa credit scoring challenge. <https://challenge.kalapa.vn/>, 2020.
- Donald Ervin Knuth. *The art of computer programming*, volume 3. Pearson Education, 1997.
- J Zico Kolter and Marcus A Maloof. Dynamic weighted majority: An ensemble method for drifting concepts. *JMLR*, 8:2755–2790, 2007.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NeurIPS*, pages 6402–6413, 2017.
- Linh Le. The 1st rank solution for kalapa credit scoring challenge. <https://github.com/klcute/kalapa/>, 2020.
- Yann LeCun. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1998.
- Stefan Lee, Senthil Purushwalkam Shiva Prakash, Michael Cogswell, Viresh Ranjan, David Crandall, and Dhruv Batra. Stochastic multiple choice learning for training diverse deep ensembles. In *NeurIPS*, pages 2119–2127, 2016.
- Zachary Lipton, Yu-Xiang Wang, and Alexander Smola. Detecting and correcting for label shift with black box predictors. In *ICML*, pages 3122–3130, 2018.
- A Rupam Mahmood, Hado P van Hasselt, and Richard S Sutton. Weighted importance sampling for off-policy learning with linear function approximation. In *NeurIPS*, pages 3014–3022, 2014.
- Robert J May, Holger R Maier, and Graeme C Dandy. Data splitting for artificial neural networks using som-based stratified sampling. *Neural Networks*, 23(2):283–294, 2010.
- Leandro L Minku, Allan P White, and Xin Yao. The impact of diversity on online ensemble learning in the presence of concept drift. *IEEE TKDE*, 22(5):730–742, 2009.

- Minlong Peng, Qi Zhang, Xiaoyu Xing, Tao Gui, Xuanjing Huang, Yu-Gang Jiang, Keyu Ding, and Zhigang Chen. Trainable undersampling for class-imbalance learning. In *AAAI*, pages 4707–4714, 2019.
- Stephan Rabanser, Stephan Günnemann, and Zachary Lipton. Failing loudly: An empirical study of methods for detecting dataset shift. In *NeurIPS*, pages 1396–1408, 2019.
- Carl Edward Rasmussen and Zoubin Ghahramani. Occam’s razor. In *NeurIPS*, pages 294–300, 2001.
- Frank Schorfheide and Kenneth I Wolpin. On the use of holdout samples for model selection. *American Economic Review*, 102(3):477–81, 2012.
- Hidetoshi Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of statistical planning and inference*, 90(2):227–244, 2000.
- Amos Storkey. When training and test sets are different: characterizing learning transfer. *Dataset shift in machine learning*, pages 3–28, 2009.
- Masashi Sugiyama, Shinichi Nakajima, Hisashi Kashima, Paul V Buenau, and Motoaki Kawanabe. Direct importance estimation with model selection and its application to covariate shift adaptation. In *NeurIPS*, pages 1433–1440, 2008.
- Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *NeurIPS*, pages 1057–1063, 2000.
- Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *ICLR*, 2014.
- Bianca Zadrozny. Learning and evaluating classifiers under sample selection bias. In *ICML*, page 114, 2004.
- Kun Zhang, Bernhard Schölkopf, Krikamol Muandet, and Zhikun Wang. Domain adaptation under target and conditional shift. In *ICML*, pages 819–827, 2013.
- Tianyi Zhou, Shengjie Wang, and Jeff A Bilmes. Diverse ensemble evolution: Curriculum data-model marriage. In *NeurIPS*, pages 5905–5916, 2018.

Appendix A. First Appendix

This is the first appendix.

Appendix B. Second Appendix

This is the second appendix.